Quick IPT — Mathematics and Implementation of Hidden-Point Measurement

A detailed companion to the EVKA Position IPT report

EVKA Position Project

July 8, 2026

1 Introduction and problem statement

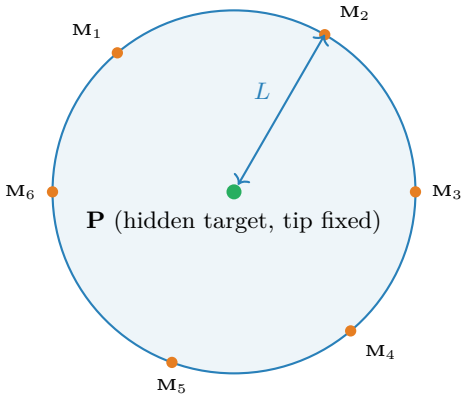
The EVKA Position system measures a Cartesian point (X, Y, Z) in millimetres from three encoders: one draw-wire encoder for radius r and two rotary encoders for azimuth θ and elevation ϕ. It reports a **3-DOF point**, not a 6-DOF pose — there is no orientation sensor. Consequently a point behind an obstacle, around a corner, or inside a recess cannot be reached by the tip directly: the tip has no line of travel to it.

Quick IPT removes this restriction with a purely geometric trick. Attach a rigid pen to the draw-wire body so that the wire couples to the pen at a fixed offset L from the pen *tip*. Hold the tip motionless on the hidden target $\mathbf{P}$ and sweep the handle through a wide cone. Because the tip does not move, the wire-side attachment point $\mathbf{M}_i$ that the sensor actually measures is always exactly L away from $\mathbf{P}$:

$$\|\mathbf{M}_i - \mathbf{P}\| = L \qquad \text{for every sample } i. \tag{1}$$

Every $\mathbf{M}_i$ therefore lies on a sphere of radius L centred on the target. Fitting that sphere to the measured point cloud recovers its centre $\mathbf{P}$ — the hidden target — and, if L is treated as unknown, the radius simultaneously *self-calibrates* the pen offset.

All measured points $\mathbf{M}_i$ lie on a sphere of radius L centred on $\mathbf{P}$



The single sample is useless on its own — $\mathbf{P}$ could be anywhere on a sphere around it — which is exactly why the method needs *many* samples spread over a wide cone. Sections 3–5 make that intuition precise: the recoverability of $\mathbf{P}$ is governed by the angular spread of the samples, and a small cone is a nearly flat patch of a large sphere that pins the centre only weakly.

2 Research context and related work

Prodim Proliner and the patent. The method was pioneered by **Prodim International** for the Proliner family of draw-wire 3D templating scanners and is protected by **US 9,267,794 B2** (Teune & Janssen, "Method of determining a target spatial coordinate using an apparatus comprising a movable hand-held probe and a portable base unit"). The patent describes precisely the two solver modes used here. With a *known* offset it forms, for each orientation, a virtual sphere "having centres corresponding to the spatial coordinates of the attachment point, wherein radii of the spheres equal a crow flying distance between the attachment point and a pointing tip," and takes the target at their intersection — this is the fixed-radius trilateration of §4.2. With an *unknown* offset it instead determines "one virtual sphere spanned by the coordinates of each attachment point," whose centre "being the target spatial coordinate" — the self-calibrating sphere fit of §4.1–4.3. EVKA Position has the same sensor topology (one wire + two angle encoders), so the method transfers directly and needs **no firmware change**.

Pivot calibration. Mathematically, IPT is identical to *pivot calibration*, a standard problem in surgical navigation and robotics: find the fixed tip of a tracked tool by pivoting it about a stationary point. Yaniv's survey "Which Pivot Calibration?" (SPIE 2015) compares three formulations — geometry-based *sphere fitting* (SF), an *algebraic one-step* (AOS), and an *algebraic two-step* (ATS) estimator — and finds that algebraic formulations seeded by, or combined with, a geometric refinement are more precise and accurate than a naive sphere fit (reporting $\sim$0.25–0.35 mm on optical trackers). Our solver follows exactly that recommended two-stage pattern: an algebraic seed (§4.1 or §4.2) followed by a Gauss–Newton geometric refinement (§4.3). The same problem appears in robotics as *tool-centre-point (TCP) calibration*.

Algebraic sphere fitting. The linear-least-squares sphere fit we seed with is the sphere analogue of Coope's circle fit ("Circle fitting by linear and nonlinear least squares," JOTA 1993): linearise the sphere equation by absorbing the quadratic centre term into a single extra unknown, solve one linear system, then optionally refine.

Dilution of precision. The dependence of the target error on the *shape* of the sweep, not just the noise level, is dilution of precision — the same phenomenon that makes a GNSS fix poor when the visible satellites are clustered in one part of the sky. We make this quantitative in §5 through the condition number of a geometric Jacobian of outward unit vectors, directly analogous to the geometry matrix behind GDOP.

3 Geometric model and formulation

Write the measured attachment points as

$$\mathbf{M}_i \;=\; \mathbf{P} + L\,\mathbf{d}_i + \boldsymbol{\varepsilon}_i, \qquad i = 1, \dots, n, \tag{2}$$

where $\mathbf{P} \in \mathbb{R}^3$ is the hidden target, $L > 0$ the (possibly unknown) pen offset, $\mathbf{d}_i \in \mathbb{R}^3$ a unit vector from tip to attachment set by the operator's hand orientation on sample i, and $\boldsymbol{\varepsilon}_i$ the sensor error. We model $\boldsymbol{\varepsilon}_i$ as zero-mean, approximately isotropic with standard deviation σ per axis; on real hardware σ bundles encoder quantisation, the exponential-moving-average output filter, and mechanical play.

Dropping the noise, (2) gives the exact constraint (1). The unknowns are $\mathbf{P}$ (three components) and L (one), so **four** in the self-calibrating case and **three** when L is supplied. Each sample contributes one scalar equation, so four points define a sphere in principle. In practice the solver requires

$$n \;\geq\; \texttt{MIN\_POINTS} = 8,$$

because a real, noisy, partial arc needs redundancy for a stable fit — four points on a small cap are catastrophically ill-conditioned (§5).

All coordinates are in the **sensor origin frame**: the boot zero-point set by `setZeroPoint()`, in millimetres, the same frame as the live $X/Y/Z$ readout. The recovered $\mathbf{P}$ inherits that frame; it is not a world coordinate.

4 The three estimators

The solver combines three classical estimators. Two are linear and closed-form and serve as seeds; the third is an iterative geometric refinement that produces the final answer and the residual used for quality gating. Each subsection ends with the function in `tools/ipt/solver.py` that implements it.

4.1 [A] Coope algebraic linear least squares

Start from the squared constraint and expand:

$$\|\mathbf{M}-\mathbf{C}\|^2 = r^2 \iff \|\mathbf{M}\|^2 - 2\,\mathbf{M}^\top\mathbf{C} + \|\mathbf{C}\|^2 = r^2. \tag{3}$$

The term $\|\mathbf{C}\|^2$ is what makes this nonlinear in $\mathbf{C}$. Coope's device is to absorb it into a single scalar unknown $g \equiv r^2 - \|\mathbf{C}\|^2$, leaving an equation that is *linear* in the four unknowns $(\mathbf{C}, g)$:

$$2\,\mathbf{M}_i^\top\mathbf{C} + g = \|\mathbf{M}_i\|^2 . \tag{4}$$

Stacking all n samples,

$$\underbrace{\begin{bmatrix} 2\mathbf{M}_1^\top & 1 \\ \vdots & \vdots \\ 2\mathbf{M}_n^\top & 1 \end{bmatrix}}_{A} \underbrace{\begin{bmatrix} \mathbf{C} \\ g \end{bmatrix}}_{\mathbf{x}} = \underbrace{\begin{bmatrix} \|\mathbf{M}_1\|^2 \\ \vdots \\ \|\mathbf{M}_n\|^2 \end{bmatrix}}_{b}, \qquad \mathbf{x} = \arg\min_{\mathbf{x}} \|A\mathbf{x} - b\|^2 , \tag{5}$$

solved by one linear least squares. The radius is recovered as $r = \sqrt{g + \|\mathbf{C}\|^2}$.

Centering. In this workspace $\|\mathbf{M}_i\| \sim 10^3$ mm, so the coordinate columns of A are $\sim 10^3$ while the constant column is 1: a scale mismatch of a thousand that inflates $\mathrm{cond}(A)$ for a reason that has nothing to do with the measurement geometry. The fix is to subtract the cloud mean $\bar{\mathbf{M}}$ before solving, so the (centred) coordinate columns and the constant column are comparable, then add $\bar{\mathbf{M}}$ back to the recovered centre. This is exactly what the code does:

```
mean = M.mean(axis=0)
Mc = M - mean
A = np.hstack([2 * Mc, np.ones((len(Mc), 1))])
b = np.sum(Mc ** 2, axis=1)
sol, _, _, sv = lstsq(A, b, rcond=None)
Cc = sol[:3]
C = Cc + mean                              # add the mean back
r = np.sqrt(max(0.0, sol[3] + Cc @ Cc))    # guard neg. radicand
```

$\Rightarrow$ `fit_sphere_algebraic(M)` $\to (\mathbf{C}, r, \mathrm{cond})$. This is the seed for the self-calibrating path, and its radius r is reported separately as `L_fit`, an *independent* estimate of the pen length that can be compared against a user-supplied L as a sanity check.

4.2 [B] Fixed-radius differencing (trilateration)

When the pen length L is known, the radius is no longer an unknown and a better conditioned linear solve is available. Expand the constraint for two samples i and 0 and subtract:

$$\|\mathbf{M}_i\|^2 - 2\,\mathbf{M}_i^\top \mathbf{P} + \|\mathbf{P}\|^2 = L^2, \tag{6}$$

$$\|\mathbf{M}_0\|^2 - 2\,\mathbf{M}_0^\top \mathbf{P} + \|\mathbf{P}\|^2 = L^2. \tag{7}$$

Both the quadratic term $\|\mathbf{P}\|^2$ *and* the (equal) radius L^2 cancel, leaving a relation linear in $\mathbf{P}$ alone:

$$2\,(\mathbf{M}_i - \mathbf{M}_0)^\top \mathbf{P} \;=\; \|\mathbf{M}_i\|^2 - \|\mathbf{M}_0\|^2\,, \qquad i = 1, \ldots, n-1. \tag{8}$$

This is trilateration: each equation says $\mathbf{P}$ is equidistant (namely L) from $\mathbf{M}_i$ and $\mathbf{M}_0$, i.e. it lies on the radical plane between the two spheres. Because the radius unknown has been eliminated, the 3×3 system is far better conditioned than (5); the value of L never even enters, only the fact that it is the *same* for every point.

```
M0 = M[0]
A = 2 * (M[1:] - M0)
d = np.sum(M[1:] ** 2, axis=1) - np.sum(M0 ** 2)
p, _, _, sv = lstsq(A, d, rcond=None)
```

$\Rightarrow$ `trilaterate_fixed_radius(M)` $\rightarrow (\mathbf{P}, \text{cond})$, needing $n \geq 4$. It is the seed for the known-L path.

4.3 [C] Gauss–Newton geometric refinement

The linear estimators [A] and [B] minimise an *algebraic* residual — the error in (4)/(8) — which weights each point by a factor that grows with its distance from the centre and is therefore biased on noisy, partial arcs. The statistically correct objective, and the one that yields the residual we report, is the *geometric* residual: the signed Euclidean distance from each point to the fitted sphere surface,

$$f_i(\mathbf{C}, r) \;=\; \|\mathbf{M}_i - \mathbf{C}\| - r. \tag{9}$$

Under isotropic Gaussian noise ε_i, minimising $\sum_i f_i^2$ is the maximum- likelihood estimate of $(\mathbf{C}, r)$. We minimise it with Gauss–Newton. Let

$$\mathbf{u}_i \;=\; \frac{\mathbf{M}_i - \mathbf{C}}{\|\mathbf{M}_i - \mathbf{C}\|} \tag{10}$$

be the outward unit vector at sample i. The partial derivatives of (9) are

$$\frac{\partial f_i}{\partial \mathbf{C}} = -\mathbf{u}_i^\top, \qquad \frac{\partial f_i}{\partial r} = -1, \tag{11}$$

so the Jacobian row is $[\,-\mathbf{u}_i^\top,\; -1]$. Each iteration solves the linearised least-squares step $J\,\Delta = -\mathbf{f}$ and updates $[\mathbf{C}; r]$ += Δ:

$$J = \begin{bmatrix} -\mathbf{u}_1^\top & -1 \\ \vdots & \vdots \\ -\mathbf{u}_n^\top & -1 \end{bmatrix} \in \mathbb{R}^{n \times 4}, \qquad \Delta = \arg\min_{\Delta} \|J\Delta + \mathbf{f}\|^2\,. \tag{12}$$

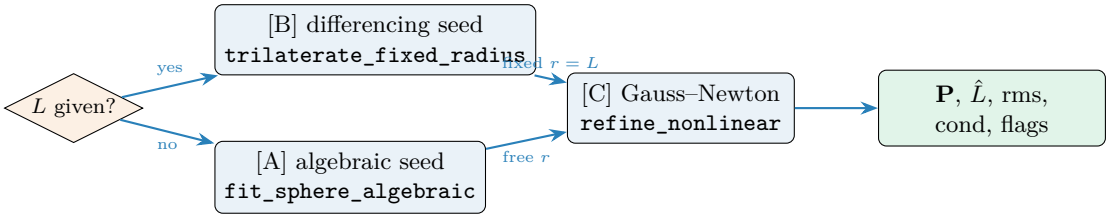
When L is known the radius is held fixed: the last column of J is dropped, the step solves for $\Delta\mathbf{C}$ only, and $J \in \mathbb{R}^{n \times 3}$. The implementation floors $\|\mathbf{M}_i - \mathbf{C}\|$ at 10^{-9} (a point exactly at the centre has no defined direction), breaks if a step exceeds 10^4 (divergence guard), and stops when $\|\Delta\| < 10^{-9}$:

```
diff = M - C
dist = np.maximum(norm(diff, axis=1), 1e-9)   # guard /0
u = diff / dist[:, None]
f = dist - r
J = np.hstack([-u, -np.ones((len(M), 1))])    # -u only, if fixed_r
dx, _, _, _ = lstsq(J, -f, rcond=None)
C = C + dx[:3];   r = r + dx[3]               # update
```

$\Rightarrow$ `refine_nonlinear(M, C0, r0=..., fixed_r=...)` $\rightarrow (\mathbf{C}, r, \text{rms})$, where the returned root-mean-square of $\{f_i\}$ is the quality metric of §6.

4.4 The pipeline: `solve_ipt`

`solve_ipt(M, L=None)` wires the three estimators into the two-stage, seed-then-refine pattern recommended by Yaniv:



- **Self-calibrating** ($L =$ `None`): algebraic seed [A] $\rightarrow$ Gauss–Newton [C] with free radius. $\hat{L}$ is the fitted pen length; it should match the physical pen within a few mm.
- **Known** L: differencing seed [B] $\rightarrow$ Gauss–Newton [C] with the radius held at L. $\hat{L} = L$ (the value used); `L_fit` is the independent algebraic radius from [A], reported so the operator can check the entered length against the data.

Because $n \geq$ `MIN_POINTS` $= 8$ is enforced first, the $n \geq 4$ precondition of [B] always holds. The return dictionary carries

$\{$ `ok`, $\mathbf{P}$, `L_hat`, `L_fit`, `rms_resid`, `cond`, `n_points`, `slip_warning`, `geom_warning` $\}$.

5 Conditioning and dilution of precision

The residual RMS tells you whether the points lie on *a* sphere; it does **not** tell you whether that sphere's centre is well determined. A small cap of a large sphere fits its points beautifully (tiny RMS) yet leaves the centre almost free to slide along the sweep axis. The quantity that captures this is the conditioning of the geometric Jacobian.

Why the algebraic condition number is the wrong indicator. Centering the cloud before the algebraic fit (§4.1) deliberately removes the scale-driven part of $\text{cond}(A)$ — which was the whole point of centering — so the algebraic condition number no longer reflects sweep geometry. Using it as a "sweep wider" cue would be meaningless.

The geometric Jacobian. Instead, use the Jacobian of outward unit vectors evaluated at the solution,

$$J = \begin{bmatrix} \mathbf{u}_1^\top & 1 \\ \vdots & \vdots \\ \mathbf{u}_n^\top & 1 \end{bmatrix} \in \mathbb{R}^{n\times 4} \quad \text{(unit vectors, plus a ones column)}, \qquad \kappa = \text{cond}(J) = \frac{\sigma_{\max}(J)}{\sigma_{\min}(J)}. \tag{13}$$

Near the solution the linearised model is $\mathbf{f} \approx J\,\delta$ with $\delta = [\delta\mathbf{C}; \delta r]$, so the parameter perturbation induced by measurement noise is $\delta \approx (J^\top J)^{-1} J^\top \boldsymbol{\varepsilon}$, giving the first-order covariance

$$\mathrm{Cov}(\hat{\mathbf{P}}) \;\approx\; \sigma^2 \left[(J^\top J)^{-1}\right]_{1:3,\,1:3}. \tag{14}$$

When the unit vectors $\mathbf{u}_i$ cluster (a narrow cone) they become nearly parallel to one another and to the constant column, $J^\top J$ approaches singularity, and $(J^\top J)^{-1}$ blows up along the sweep axis. κ is a scalar summary of that blow-up: a **dilution-of-precision** number, mathematically the same geometry factor as the GDOP of a GNSS fix (unit vectors to satellites) or the condition of a pivot-calibration design matrix.

The same 4-column form (13) is used for both the self-calibrating and known-L paths so the thresholds stay consistent; it is monotone with angular spread either way.

```
u = diff / np.maximum(norm(diff, axis=1), 1e-9)[:, None]
J = np.hstack([u, np.ones((len(M), 1))])
return float(np.linalg.cond(J))                    # geometry_cond(M, C)
```

Averaging. For a fixed geometry, (14) scales as σ^2/n through the $J^\top J$ accumulation, so the target error falls roughly as $\sigma/\sqrt{n}$. More points average down noise; they do *not* fix bad geometry — only a wider sweep does. Section 8 confirms both trends numerically.

6 Quality gates

Two independent, heuristic gates translate the fit into an operator verdict. They are deliberately generous: the wire encoder plus EMA filter is coarser than the sub-mm optical trackers of the pivot-calibration literature (Yaniv reports 0.3–0.8 mm there), so mm-scale bands are appropriate and should be re-tuned on real hardware.

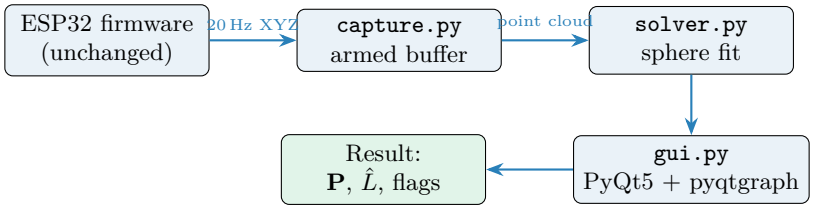
Flag	Metric	Bands
slip_warning	RMS geometric residual	`ok` < 2 mm, `warn` 2–5 mm, `reject` > 5 mm
geom_warning	Geometric Jacobian κ	`ok` < 150, `warn` 150–1500, `block` > 1500

- **RMS residual** (`RESIDUAL_ACCEPT_MM` $= 2$, `RESIDUAL_REJECT_MM` $= 5$) rises when the tip slips off the target during the sweep, because the points then no longer lie on a single sphere. It flags slip but cannot correct it — re-measure.
- **Geometry** κ (`COND_WARN` $= 150$, `COND_BLOCK` $= 1500$) rises when the sweep is too small. As §8 shows, $\kappa = 150$ corresponds to $\sim 18°$ of half-angle (~ 1.6 mm error) and $\kappa = 1500$ to $\sim 5°$ (~ 20 mm error).

The GUI combines them into one verdict: `reject` if either `slip` = `reject` or `geom` = `block`; else `warn` if either warns; else `ok`.

7 Implementation

The entire feature is a host-side Python package, `tools/ipt/`; the ESP32 firmware is untouched. The tool subscribes to the existing 20 Hz position stream, buffers a swept point cloud, and runs `solve_ipt` on demand.



7.1 Transport asymmetry and the pairing state machine

The firmware emits different line formats on the two transports:

- **Serial** — one self-contained line per cycle, carrying position and validity together:

  $$\texttt{DATA},x,y,z,r,\theta,\phi,\texttt{is\_valid},\texttt{frame\_count},\texttt{ts\_ms}$$

  parsed by `parse_line` into a `ParsedFrame`; non-finite fields make the parse return `None`, and `DataStore.push` drops any frame with `is_valid==0` before it reaches the buffer.

- **TCP** — position and validity arrive as *two separate lines*:

  $$\texttt{X}x,\texttt{Y}y,\texttt{Z}z \quad \texttt{then} \quad \texttt{SENSOR},r,\theta,\phi,\texttt{is\_valid},\texttt{frame\_count}$$

  The `X..,Y..,Z..` line has the coordinates but no validity; the following `SENSOR,..` line has validity but no coordinates.

The capture buffer therefore runs a tiny two-state pairing machine on the TCP path: an XYZ line is *stashed* (committing nothing), and the next SENSOR line consumes the stash, committing the point through the acceptance gate only if `is_valid` is set. Any intervening non-XYZ/non-SENSOR line leaves the stash untouched; `disarm()`/`clear()` reset it so a half-received pair never straddles a re-arm.

```
xyz = parse_xyz_line(line)
if xyz is not None:
    self._pending_xyz = (xyz.x_mm, xyz.y_mm, xyz.z_mm)   # stash; commit later
    return False
sensor = parse_sensor_line(line)
if sensor is not None:
    pending = self._pending_xyz
    self._pending_xyz = None                        # consume the slot
    if sensor.is_valid and pending is not None:
        return self._accept(*pending)               # commit paired point
return False
```

7.2 Acceptance gates

Every candidate point passes three gates in `IptCapture._accept` before it enters the cloud:

1. **Armed** — reject unless capture is armed.
2. **Finite** — reject any non-finite coordinate (belt-and-braces on the TCP path, where floats are parsed straight off the wire).
3. **Dedup** — reject if the squared displacement from the last accepted point is below $\texttt{DEFAULT\_MIN\_DISP\_MM}^2 = (1\,\text{mm})^2$. At 20 Hz a momentarily still handle emits many near-identical frames; only points that moved ≥ 1 mm are kept. The threshold is stored squared to avoid a square root per sample.

7.3 GUI and threading

`gui.py` (`IptWindow`) renders three 2D projections (top XY, front XZ, side YZ) using `pyqtgraph` rather than a 3D GL view, for portability on headless / Wayland / VM setups. Transports run on daemon threads that only enqueue raw lines (TCP) or push parsed frames to a thread-safe `DataStore` (serial); a 40 ms (25 Hz) `QTimer` on the GUI thread drains the queue, feeds the capture buffer, and redraws. All buffer mutation and all plotting happen on the GUI thread — the capture methods are never called from a transport callback. The operator flow is **CONNECT** → **ARM** → **sweep** → **STOP** → **SOLVE**; SOLVE draws the fitted circle of radius $\hat{L}$ on each projection and an orange marker at **P**, and prints the two quality flags.

7.4 Solver ↔ mathematics

Function (`solver.py`)	Section	Role
`fit_sphere_algebraic`	§4.1	[A] Coope linear seed; also gives `L_fit`
`trilaterate_fixed_radius`	§4.2	[B] known-L differencing seed
`refine_nonlinear`	§4.3	[C] Gauss–Newton; returns final $\mathbf{P}$ and RMS
`geometry_cond`	§5	DOP indicator κ
`solve_ipt`	§4.4	pipeline + gating
`sample_directions`	§8	synthetic golden-spiral sweep (demo/tests/study)

7.5 Self-check and tests

Running `python -m tools.ipt.solver` synthesises 12 points on a 35° golden spiral at $L = 400\,\text{mm}$ with 0.1 mm noise and prints, reproducibly, $\hat{\mathbf{P}} = [1200.101, -450.051, 300.389]$, $\hat{L} = 399.596\,\text{mm}$, rms $= 0.073\,\text{mm}$, $\kappa = 35$, both flags `ok`, and a target error of 0.405 mm — proving the math before any hardware is attached. The offline pytest suite (`tools/ipt/tests/`) asserts, among others: self-cal target error $< 2\,\text{mm}$ and $\hat{L}$ within 1 mm; the known-L path within 2 mm; that a 5 mm tip wander raises `slip_warning`; that a 2° sweep raises `geom_warning`; that $n = 7$ returns `ok:False` without crashing; the 1 mm dedup boundary; and the TCP pairing with validity recovery.

8 Simulation study

To quantify the analysis of §5, `docs/reports/ipt/sim_study.py` runs the *actual* `solve_ipt` on synthetic clouds. Ground truth is $\mathbf{P}_{\text{true}} = [1200, -450, 300]\,\text{mm}$ and $L_{\text{true}} = 400\,\text{mm}$; sweeps use the same golden-spiral `sample_directions` the operator's hand motion is modelled on; each configuration is averaged over 16 independent noise realisations. The reported error is $\left\|\hat{\mathbf{P}} - \mathbf{P}_{\text{true}}\right\|$, self-calibrating (i.e. L treated as unknown).

8.1 Error and conditioning versus sweep half-angle

Half-angle	κ (cond)	RMS (mm)	Target err (mm)	$\hat{L}$ err (mm)
2°	22684	0.076	181.751	181.689
3°	5549	0.076	59.490	59.433
5°	1850	0.076	20.490	20.436
8°	717	0.076	7.968	7.915
12°	317	0.076	3.554	3.500
18°	140	0.076	1.595	1.547
25°	71	0.076	0.836	0.789
35°	35	0.076	0.436	0.386
50°	16	0.077	0.225	0.169
70°	8	0.078	0.129	0.076

Table 1: Target error, self-calibrated length error, sphere-fit RMS and geometric condition number versus sweep half-angle ($n = 12$, $\sigma = 0.1\,\text{mm}$, 16 seeds). Note the RMS is essentially flat — the points always fit a sphere well — while the target error spans three orders of magnitude. RMS alone is blind to the geometry; κ tracks it.

Table 1 is the central result. The residual RMS barely moves (0.076–0.078 mm) across the whole range, yet the target error collapses from 182 mm at a 2° pinhole sweep to 0.13 mm at a generous 70° cone. This is dilution of precision made concrete: a good-looking fit (small RMS)

can still hide a badly located centre, and only κ exposes it. The gate thresholds land exactly where intended — $\kappa = \texttt{COND\_BLOCK} = 1500$ at $\sim 5°$ ($\approx 20\,\text{mm}$ error) and $\kappa = \texttt{COND\_WARN} = 150$ at $\sim 18°$ ($\approx 1.6\,\text{mm}$ error).

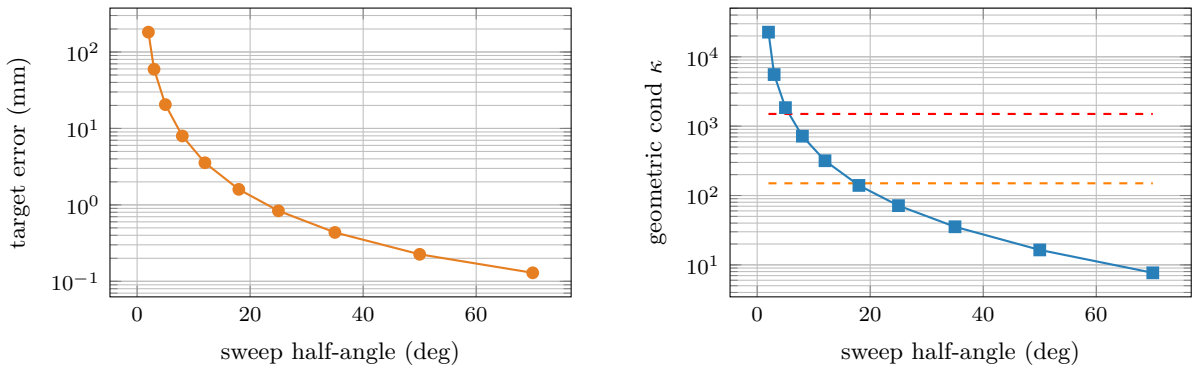


Figure 1: Left: target error (log scale) versus half-angle — a small sweep is catastrophic. Right: geometric condition number κ (log scale) with the `COND_BLOCK`= 1500 (red) and `COND_WARN`= 150 (orange) gate lines.

8.2 Error versus noise and point count

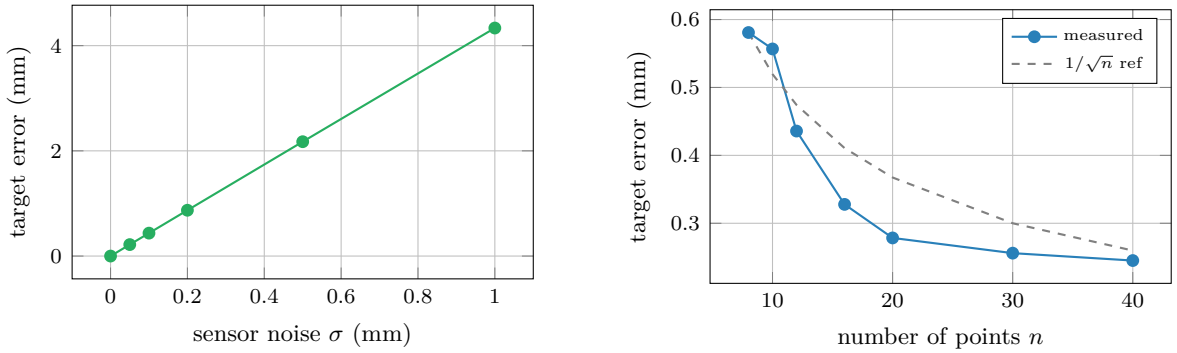


Figure 2: Left: target error is *linear* in the sensor noise σ (half-angle 35°, $n = 12$) — doubling the noise doubles the error, as (14) predicts. Right: target error versus point count (half-angle 35°, $\sigma = 0.1\,\text{mm}$) against a $1/\sqrt{n}$ reference; more samples average the noise down but with diminishing returns.

Figure 2 confirms the two predictions of §5: for fixed geometry the error is proportional to σ (left), and it decreases roughly as $1/\sqrt{n}$ with the number of points (right). The practical lesson is the same as the analysis: **a wider sweep is worth far more than more points**. Going from 12 to 40 points at 35° cuts the error by about half; going from 12° to 35° at 12 points cuts it by a factor of eight.

The study script ends with a self-check `assert` (mean target error $< 2\,\text{mm}$ and $\kappa < 60$ at 35°, both flags `ok`) so a regression in the solver or its numbers fails loudly. Regenerate all figures and tables with `python docs/reports/ipt/sim_study.py`.

9 Limitations

- **3-DOF only.** EVKA Position returns a point, not a pose. IPT recovers the hidden *point*, never the 6-DOF orientation of the pen.
- **Tip slip.** The RMS residual flags it (§6) but cannot correct it. If the tip wanders off the target, the points stop lying on one sphere; re-measure.

- **Sweep geometry.** Small or single-plane sweeps are ill-conditioned (Table 1). The `geom_warning` flag is the operator's cue to open the cone; in tight spaces a two-direction "cross" motion is the minimum viable substitute for a full spiral.
- **Coordinate frame.** Results are in the sensor boot zero-point frame, not a world frame. Re-zeroing or moving the sensor invalidates a previously recovered target; transform externally if world coordinates are needed.
- **Heuristic gates.** The 2/5 mm and 150/1500 thresholds were tuned in simulation against a 0.1 mm noise model. Real hardware noise is larger and should be characterised, and the gates re-tuned, before field use.

10 Conclusion

Quick IPT turns a 3-DOF draw-wire measuring system into a hidden-point probe with no firmware change, by exploiting a single geometric fact: samples taken while the tip is fixed lie on a sphere centred on the target. The estimator is a textbook seed-then-refine pipeline — an algebraic linear fit (Coope) or a fixed-radius differencing solve for the seed, followed by a Gauss–Newton minimisation of the geometric residual — identical in structure to surgical and robotic pivot calibration. The dominant error source is not sensor noise but sweep geometry: the simulation study shows the target error spanning three orders of magnitude across half-angle while the fit residual stays flat, which is exactly why the tool gates on the condition number of a geometric Jacobian rather than on the residual alone. Sweep wide.

Intellectual property note

The Quick IPT method is patented by Prodim International (US 9,267,794 B2, EP 2,792,994 B1). This implementation is a technical reverse-engineering exercise for research and educational purposes. The underlying mathematics — sphere fitting, trilateration, and pivot calibration — is well established in the literature. Commercial deployment may require licensing from Prodim.

References

[1] R. Teune and A. J. Janssen, "Method of determining a target spatial coordinate using an apparatus comprising a movable hand-held probe and a portable base unit, and a related apparatus," US Patent 9,267,794 B2, 2016 (EP 2,792,994 B1).

[2] Z. Yaniv, "Which Pivot Calibration?," *Proc. SPIE Medical Imaging*, vol. 9415, 941527, 2015.

[3] I. D. Coope, "Circle fitting by linear and nonlinear least squares," *J. Optim. Theory Appl.*, vol. 76, no. 2, pp. 381–388, 1993.

[4] E. C. S. Chen *et al.*, "Is pose-based pivot calibration superior to sphere fitting?," *Proc. SPIE Medical Imaging*, vol. 10135, 2017.

[5] R. B. Langley, "Dilution of precision," *GPS World*, vol. 10, no. 5, pp. 52–59, 1999.